មេរៀនទី១

សេចក្តីផ្តើម និងមូលដ្ឋាន C#

(Introduction & C# Basics)

មុខវិជ្ជា៖ ការសរសេរកម្មវិធី C# ភាគ១

លេខកូដ: CS-101

កម្រិតសិក្សា: បរិញ្ញាបត្រ

ឆ្នាំសិក្សា: ២០២៥-២០២៦

មាតិកា (Table of Contents)

- ១.១ សេចក្តីផ្តើមអំពីការសរសេរកម្មវិធី
- ១.២ ប្រវត្តិ និងទិដ្ឋភាពទូទៅនៃ C#
- ១.៣ .NET Framework និងបរិស្ថានអភិវឌ្ឍ
- ១.៤ រចនាសម្ព័ន្ធកម្មវិធី និងវាក្យសម្ព័ន្ធ C#
- ១.៥ Console Input និង Output
- ១.៦ ប្រភេទទិន្នន័យ និងអថេរ
- ១.៧ ប្រតិបត្តិករ និង Expressions

១.១ សេចក្តីផ្តើមអំពីការសរសេរកម្មវិធី (Introduction to Programming)

ក. អ្វីទៅជាការសរសេរកម្មវិធី (What is Programming?)

ការសរសេរកម្មវិធី (Programming) គឺជាដំណើរការនៃការបង្កើតសំណុំនៃការណែនាំ (instructions) ដែលប្រាប់ឱ្យកុំព្យូទ័រធ្វើកិច្ចការជាក់លាក់មួយ។ កម្មវិធីកុំព្យូទ័រ (Computer Programs) គឺជាសំណុំនៃការណែនាំ ទាំងនោះដែលត្រូវបានសរសេរជាភាសាកម្មវិធី (Programming Language) ដែលកុំព្យូទ័រអាចយល់បាន។

យើងអាចស្រមៃមើលការសរសេរកម្មវិធីដូចជាការសរសេររូបមន្តធ្វើម្ហូបមួយ។ នៅក្នុងរូបមន្ត យើងបានរៀបរាប់ជាជំហាន ទាំងអស់ដែលចាំបាច់ក្នុងការធ្វើម្ហូប ស្រដៀងគ្នានេះដែរ នៅក្នុងកម្មវិធីកុំព្យូទ័រ យើងសរសេរជំហានទាំងអស់ ដែលកុំព្យូទ័រត្រូវធ្វើតាម។

គោលការណ៍មូលដ្ឋាននៃការសរសេរកម្មវិធី:

- ការវិភាគបញ្ហា (Problem Analysis): យល់ពីបញ្ហាដែលត្រូវដោះស្រាយ
- ការរចនា (Design): ផែនការពីរបៀបដោះស្រាយបញ្ហា
- ការសរសេរកូដ (Coding): សរសេរកម្មវិធីដោយប្រើភាសាកម្មវិធី
- ការធ្វើតេស្ត (Testing): ពិនិត្យមើលថាកម្មវិធីដំណើរការត្រឹមត្រូវ
- ការថែទាំ (Maintenance): កែលំអ និងធ្វើឱ្យប្រសើរឡើង

ខ. ប្រភេទភាសាកម្មវិធី (Types of Programming Languages)

មានភាសាកម្មវិធីជាច្រើនប្រភេទ ដែលអាចចាត់ថ្នាក់តាមកម្រិតនៃភាពស្មុគស្មាញ និងវិធីសាស្ត្រនៃការធ្វើការ។

១. ភាសាកម្រិតទាប (Low-Level Languages):

ភាសាទាំងនេះជិតទៅនឹងភាសាម៉ាស៊ីន (Machine Language) ហើយផ្តល់នូវការគ្រប់គ្រងផ្ទាល់លើហាតវែរ។ ឧទាហរណ៍៖ Assembly Language, Machine Code

២. ភាសាកម្រិតខ្ពស់ (High-Level Languages):

ភាសាទាំងនេះងាយស្រួលប្រើប្រាស់សម្រាប់មនុស្ស ហើយត្រូវបានបកប្រែទៅជាភាសាម៉ាស៊ីន។ ឧទាហរណ៍៖ C#, Java, Python, JavaScript, C++, Ruby

៣. ភាសាតាមការិយាល័យ (Paradigms):

- ភាសា Procedural: C, Pascal - មានការរៀបចំជាទម្រង់នីតិវិធី (Procedures)
- ភាសា Object-Oriented: C#, Java, Python - ផ្អែកលើគំនិតនៃវត្ថុ (Objects)
- ភាសា Functional: Haskell, Lisp - ផ្អែកលើមុខងារគណិតវិទ្យា
- ភាសា Scripting: JavaScript, PHP - សម្រាប់ធ្វើស្វ័យប្រវត្តិកម្មកិច្ចការ

គ. ការប្រើប្រាស់ C# ក្នុងការអភិវឌ្ឍកម្មវិធី

C# (ត្រូវបានបញ្ចេញសំឡេងថា "C Sharp") គឺជាភាសាកម្មវិធីកម្រិតខ្ពស់មួយដែលមានភាពទំនើប និងទទួលបានការគាំទ្រយ៉ាងល្អ។ វាត្រូវបានអភិវឌ្ឍដោយក្រុមហ៊ុន Microsoft និងត្រូវបានប្រើប្រាស់យ៉ាងទូលំទូលាយនៅក្នុងឧស្សាហកម្ម។

អត្ថប្រយោជន៍នៃការប្រើ C#:

- សាមញ្ញ និងងាយស្រួលរៀន - វាក្យសម្ព័ន្ធច្បាស់លាស់ និងមានលក្ខណៈទំនើប
- Object-Oriented - គាំទ្រគោលការណ៍ OOP យ៉ាងពេញលេញ
- Type-Safe - ការពារកំហុសកម្មវិធីតាមរយៈការត្រួតពិនិត្យប្រភេទទិន្នន័យ
- មានបណ្ណាល័យដ៏សម្បូរបែប - .NET Framework ផ្តល់នូវមុខងារជាច្រើន
- Cross-Platform - អាចដំណើរការលើ Windows, Linux, macOS តាមរយៈ .NET Core
- សហគមន៍ធំ - មានធនធានសិក្សា និងការគាំទ្រជាច្រើន

វិស័យប្រើប្រាស់ C#:

C# អាចត្រូវបានប្រើប្រាស់ក្នុងការអភិវឌ្ឍ៖

- កម្មវិធី Desktop Applications (Windows Forms, WPF)
- កម្មវិធី Web Applications (ASP.NET, Blazor)
- កម្មវិធី Mobile Applications (Xamarin, MAUI)
- ហ្គេម (Unity Game Engine)
- Cloud Applications (Azure)
- Web APIs និង Services
- Machine Learning Applications (ML.NET)

១.២ ប្រវត្តិ និងទិដ្ឋភាពទូទៅនៃ C#(History and Overview of C#)

ក. ប្រវត្តិនៃភាសា C#

ភាសា C# ត្រូវបានបង្កើតឡើងដោយ Anders Hejlsberg និងក្រុមការងាររបស់គាត់នៅក្នុងក្រុមហ៊ុន Microsoft ក្នុងអំឡុងពេលចុងទសវត្សរ៍ឆ្នាំ 1990។ វាត្រូវបានបញ្ចេញជាផ្លូវការនៅឆ្នាំ 2000 ជាមួយនឹង .NET Framework 1.0។

គោលបំណងនៃការបង្កើត C# គឺដើម្បីផ្តល់នូវភាសាកម្មវិធីដែលងាយស្រួលប្រើប្រាស់ សាមញ្ញ មានសុវត្ថិភាព និងទំនើប ដោយរួមបញ្ចូលគ្នានូវភាពល្អៗពី Java និង C++។

ខ. វិឌ្ឍន៍នៃ C#

ភាសា C# បានឆ្លងកាត់ការវិឌ្ឍន៍ជាច្រើនជំហាន៖

- C# 1.0 (2002) - កំណែដំបូង រួមជាមួយ .NET Framework 1.0
- C# 2.0 (2005) - បន្ថែម Generics, Anonymous methods
- C# 3.0 (2007) - LINQ, Lambda expressions, Auto-properties
- C# 4.0 (2010) - Dynamic binding, Named parameters
- C# 5.0 (2012) - Async/await programming
- C# 6.0 (2015) - String interpolation, Null-conditional operators
- C# 7.0 (2017) - Tuples, Pattern matching, Local functions
- C# 8.0 (2019) - Nullable reference types, async streams
- C# 9.0 (2020) - Records, Init-only properties
- C# 10.0 (2021) - Global usings, File-scoped namespaces
- C# 11.0 (2022) - Raw string literals, Required members
- C# 12.0 (2023) - Primary constructors, Collection expressions

គ. ការប្រៀបធៀប C# ជាមួយភាសាផ្សេងៗ

C# និង Java:

ទាំងពីរមានលក្ខណៈស្រដៀងគ្នាច្រើន ពីព្រោះ C# ត្រូវបានបង្កើតឡើងដោយមានឥទ្ធិពលពី Java។ ទោះជាយ៉ាងណាក៏ដោយ C# មានលក្ខណៈពិសេសទំនើបជាងដូចជា properties, delegates, events, LINQ។

C# និង C++:

ទោះបីជា C# ប្រើវាក្យសម្ព័ន្ធស្រដៀងនឹង C++ ក៏ដោយ ក៏ C# មានភាពសាមញ្ញជាង និងមានការគ្រប់គ្រង សតិសារ (Memory Management) ដោយស្វ័យប្រវត្តិតាមរយៈ Garbage Collection។

C# និង Python:

Python សាមញ្ញជាង និងងាយស្រួលរៀនសម្រាប់អ្នកចាប់ផ្តើម តែ C# មានប្រសិទ្ធភាពខ្ពស់ជាង និងផ្តល់នូវការគ្រប់គ្រងប្រភេទទិន្នន័យដែលមានភាពតឹងរឹង (Strongly-typed)។

១.៣ .NET Framework និងបរិស្ថានអភិវឌ្ឍ (Development Environment)

ក. ទិដ្ឋភាពទូទៅ .NET Framework និង .NET Core

.NET Framework គឺជាវេទិកាអភិវឌ្ឍកម្មវិធីមួយដែលផ្តល់នូវបណ្ណាល័យ (libraries) និងសេវាកម្មជាច្រើនសម្រាប់អ្នកអភិវឌ្ឍន៍។ វាត្រូវបានបង្កើតឡើងដោយ Microsoft។

សមាសធាតុសំខាន់ៗនៃ .NET:

- Common Language Runtime (CLR) - ម៉ាស៊ីនវត្ថុនិម្មិត (Virtual Machine) សម្រាប់ដំណើរការកម្មវិធី
- Base Class Library (BCL) - បណ្ណាល័យមូលដ្ឋានផ្តល់មុខងារទូទៅ
- Framework Class Library (FCL) - បណ្ណាល័យទូលំទូលាយសម្រាប់កម្មវិធីផ្សេងៗ
- Common Type System (CTS) - ប្រព័ន្ធប្រភេទទិន្នន័យរួម
- Common Language Specification (CLS) - ការកំណត់ភាសារួម

.NET Framework vs .NET Core vs .NET 5+:

.NET Framework: កំណែដើម ដំណើរការតែលើ Windows

.NET Core: កំណែ cross-platform អាចដំណើរការលើ Windows, Linux, macOS

.NET 5+: ការរួមបញ្ចូលគ្នានៃ .NET Framework និង .NET Core ដើម្បីបង្កើតវេទិកា unified

ខ. ការដំឡើង Visual Studio

Visual Studio គឺជា Integrated Development Environment (IDE) ដ៏ល្អបំផុតសម្រាប់អភិវឌ្ឍកម្មវិធី C#។ វាផ្តល់នូវឧបករណ៍ជាច្រើនសម្រាប់សរសេរកូដ debug និងធ្វើតេស្តកម្មវិធី។

ជំហានដំឡើង Visual Studio:

1. ចូលទៅកាន់គេហទំព័រ https://visualstudio.microsoft.com/
2. ទាញយក Visual Studio Community (ឥតគិតថ្លៃសម្រាប់សិស្ស និងអ្នកប្រើប្រាស់ផ្ទាល់ខ្លួន)
3. ដំណើរការ installer ហើយជ្រើសរើស ".NET desktop development" workload
4. រង់ចាំដំណើរការដំឡើងបញ្ចប់
5. ចាប់ផ្តើម Visual Studio និងចុះឈ្មោះដោយប្រើគណនី Microsoft (optional)

ជម្រើសផ្សេងទៀតសម្រាប់អភិវឌ្ឍន៍ C#:

- Visual Studio Code - កម្មវិធីសរសេរកូដទម្ងន់ស្រាល cross-platform
- JetBrains Rider - IDE ផ្សេងទៀតដែលមានអនុភាព
- MonoDevelop - ជម្រើសបើកចំហ (open-source)

គ. ការបង្កើត C# Project ដំបូង

ជំហានបង្កើត Console Application នៅក្នុង Visual Studio:

1. បើក Visual Studio
2. ជ្រើសរើស "Create a new project"
3. ជ្រើសរើស "Console App (.NET Core)" ឬ "Console App"
4. បញ្ចូលឈ្មោះ project (ឧទាហរណ៍៖ HelloWorld)
5. ជ្រើសរើសទីតាំងរក្សាទុក project
6. ចុច "Create"

បន្ទាប់ពីបង្កើតរួច Visual Studio នឹងបង្កើតកូដដំបូងដូចខាងក្រោម:

```
using System;
namespace HelloWorld
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World");
            Console.ReadLine();
        }
    }
}
```

ការដំណើរការកម្មវិធី:

- ចុចប៊ូតុង "Start" (ឬចុច F5) ដើម្បីដំណើរការជាមួយ debugger
- ឬចុច Ctrl+F5 ដើម្បីដំណើរការដោយគ្មាន debugger
- Console window នឹងបង្ហាញសារ "Hello World!"

១.៤ រចនាសម្ព័ន្ធកម្មវិធី និងវាក្យសម្ព័ន្ធ C#(Program Structure & Syntax)

ក. រចនាសម្ព័ន្ធកម្មវិធី C#

កម្មវិធី C# មានរចនាសម្ព័ន្ធមូលដ្ឋានដូចខាងក្រោម:

```
using System;                // 1. Using directives
namespace MyApplication    // 2. Namespace
{
    class Program            // 3. Class
    {
        static void Main(string[] args)  // 4. Main method
        {
            // 5. Code statements
            Console.WriteLine("Hello, C#!");
        }
    }
}
```

ការពន្យល់ផ្នែកនីមួយៗ:

- Using directives: នាំចូល (import) namespace ដែលត្រូវការ
- Namespace: ធ្វើឱ្យកូដមានការរៀបចំ និងជៀសវាងការប៉ះទង្គិចនៃឈ្មោះ

- Class: ផ្ទុកកូដកម្មវិធី C# ទាំងអស់ត្រូវតែស្ថិតនៅក្នុង class
- Main method: ចំណុចចាប់ផ្តើម (entry point) នៃកម្មវិធី
- Code statements: ការណែនាំដែលកម្មវិធីត្រូវប្រតិបត្តិ

ខ. Namespaces និង Using Directives

Namespace គឺជាវិធីរៀបចំកូដ និងជៀសវាងការប៉ះទង្គិចនៃឈ្មោះ។ វាដូចជាថតឯកសារ (folder) សម្រាប់កូដ។

```
// Using directive
using System;
using System.Collections.Generic;
using System.Linq;

// បង្កើត namespace ផ្ទាល់ខ្លួន
namespace MyCompany.MyProject
{
    class MyClass
    {
        // Code here
    }
}
```

ការប្រើប្រាស់ using directives ផ្តល់អត្ថប្រយោជន៍:

- មិនចាំបាច់សរសេរឈ្មោះពេញលេញ (fully qualified names)
- ធ្វើឱ្យកូដងាយអាន និងសាមញ្ញជាង
- អនុញ្ញាតឱ្យប្រើប្រាស់ classes ពី namespace ផ្សេង

គ. Main Method

Main method គឺជាចំណុចចាប់ផ្តើមនៃកម្មវិធី C# ណាមួយ។ នៅពេលដំណើរការកម្មវិធី CLR នឹងស្វែងរក និងប្រតិបត្តិ Main method។

ទម្រង់នៃ Main method:

```
// ទម្រង់ទី១ - មិនមាន return value និង parameters
static void Main()
{
    // Code
}

// ទម្រង់ទី២ - មាន parameters
static void Main(string[] args)
```

```
{
    // Code
}

// ទម្រង់ទី៣ - មាន return value
static int Main()
{
    // Code
    return 0;  // 0 = success
}

// ទម្រង់ទី៤ - មានទាំង return value និង parameters
static int Main(string[] args)
{
    // Code
    return 0;
}

Command - line arguments:
static void Main(string[] args)
{
    Console.WriteLine("ចំនួន arguments: " + args.Length);

    foreach (string arg in args)
    {
        Console.WriteLine(arg);
    }
}
```

ឃ. Comments និង Coding Standards

Comments គឺជាកំណត់ចំណាំក្នុងកូដដែលមិនត្រូវបានប្រតិបត្តិ។ វាជួយពន្យល់កូដ។

ប្រភេទ Comments នៅក្នុង C#:

```
// Single-line comment
// នេះគឺជា comment មួយបន្ទាត់

/*
  Multi-line comment
  នេះគឺជា comment ច្រើនបន្ទាត់
  អាចមានច្រើនបន្ទាត់
*/

/// <summary>
/// XML Documentation comment
/// ប្រើសម្រាប់បង្កើតឯកសារ API
/// </summary>
/// <param name="name">ឈ្មោះអ្នកប្រើ</param>
/// <returns>សារស្វាគមន៍</returns>
```

```
public string Greet(string name)
{
    return "Hello, " + name;
}
```

Coding Standards និង Best Practices:

- ប្រើឈ្មោះមានន័យ (meaningful names) សម្រាប់ variables និង methods
- ប្រើ PascalCase សម្រាប់ classes និង methods (ឧទាហរណ៍: MyClass, GetValue)
- ប្រើ camelCase សម្រាប់ variables និង parameters (ឧទាហរណ៍: myVariable, userName)
- សរសេរ comments សម្រាប់ពន្យល់កូដស្មុគស្មាញ
- រក្សាកូដឱ្យមានការចង្អុលបង្ហិល (indentation) ត្រឹមត្រូវ
- ដាក់ដាច់ lines សម្រាប់ធ្វើឱ្យកូដងាយអាន
- ជៀសវាងការសរសេរ methods វែងពេក (រក្សាឱ្យខ្លី និងផ្តោតតែមួយភារកិច្ច)

១.៥ Console Input និង Output

ក. Console.WriteLine( )

Console.WriteLine( ) ប្រើសម្រាប់បង្ហាញទិន្នន័យនៅលើ console ហើយប្តូរទៅបន្ទាត់ថ្មី។

```
Console.Write() vs Console.WriteLine():
// Console.Write() មិនប្តូរបន្ទាត់
Console.Write("Hello ");
Console.Write("World");  // Output: Hello World

// Console.WriteLine() ប្តូរបន្ទាត់
Console.WriteLine("Hello");
Console.WriteLine("World");
// Output:
// Hello
// World
```

ខ. Console.ReadLine( )

Console.ReadLine( ) ប្រើសម្រាប់អាន input ពីអ្នកប្រើប្រាស់។ វាអាន text មួយបន្ទាត់ហើយ return ជា string។

```
using System;
class Program
{
   static void Main( )
   {
```

```
        // សួរអ្នកប្រើឈ្មោះ
        Console.Write("សូមបញ្ចូលឈ្មោះរបស់អ្នក: ");
        string name = Console.ReadLine( );

        // សួរអាយុ
        Console.Write("សូមបញ្ចូលអាយុ: ");
        string ageInput = Console.ReadLine( );
        int age = int.Parse(ageInput);  // បម្លែងពី string ទៅ int

        // បង្ហាញលទ្ធផល
        Console.WriteLine("សួស្តី " + name + "!");
        Console.WriteLine("អ្នកមានអាយុ " + age + " ឆ្នាំ។");
    }
}
```

ការបម្លែង Input:

```
// ReadLine( ) return string ជានិច្ច
// ត្រូវបម្លែងប្រសិនចង់បានប្រភេទផ្សេង

// String to Integer
string input = Console.ReadLine( );
int number = int.Parse(input);
// ឬប្រើ Convert
int number2 = Convert.ToInt32(input);

// String to Double
double price = double.Parse(Console.ReadLine( ));

// String to Boolean
bool isTrue = bool.Parse(Console.ReadLine( ));

// ការប្រើ TryParse (សុវត្ថិភាពជាង)
string input = Console.ReadLine( );
int number;
```

```
if (int.TryParse(input, out number))
{
    Console.WriteLine("លេខត្រឹមត្រូវ: " + number);
}
else
{
    Console.WriteLine("Input មិនត្រឹមត្រូវ!");
}
```

គ. Output Formatting

មានវិធីជាច្រើនក្នុងការធ្វើទម្រង់ output:

១. String Concatenation:

```
string name = "សុខា";
int age = 20;
Console.WriteLine("ឈ្មោះ: " + name + ", អាយុ: " + age);
```

២. Composite Formatting:

```
string name = "សុខា";
int age = 20;

Console.WriteLine("ឈ្មោះ: {0}, អាយុ: {1}", name, age);
```

៣. String Interpolation (C# 6.0+):

```
string name = "សុខា";
int age = 20;
Console.WriteLine($"ឈ្មោះ: {name}, អាយុ: {age}");

// អាចមានការគណនា
Console.WriteLine($"អាយុនៅ 5 ឆ្នាំទៀត: {age + 5}");
```

៤. Format Specifiers:

```
double price = 1234.5678;

// លេខទសភាគ
Console.WriteLine($"តម្លៃ: {price:F2}");  // 1234.57 (2 decimals)
```

```
// រូបិយប័ណ្ណ
Console.WriteLine($"តម្លៃ: {price:C}");   // $1,234.57

// ភាគរយ
double percent = 0.85;
Console.WriteLine($"ភាគរយ: {percent:P}"); // 85.00%

// លេខពាន់
int number = 1234567;
Console.WriteLine($"លេខ: {number:N}");    // 1,234,567.00
```

ឧទាហរណ៍ពេញលេញ:

```
using System;

class Program
{
    static void Main( )
    {
        // Input
        Console.WriteLine("=== ព័ត៌មានសិស្ស ===");

        Console.Write("ឈ្មោះ: ");
        string name = Console.ReadLine( );

        Console.Write("អាយុ: ");
        int age = int.Parse(Console.ReadLine( ));

        Console.Write("ពិន្ទុ: ");
        double score = double.Parse(Console.ReadLine( ));

        // Output
        Console.WriteLine("\n=== សង្ខេប ===");
        Console.WriteLine($"សិស្ស: {name}");
```

```
        Console.WriteLine($"អាយុ: {age} ឆ្នាំ");
        Console.WriteLine($"ពិន្ទុ: {score:F2}");

        // ការវិភាគ
        string grade;
        if (score >= 90)
            grade = "A";
        else if (score >= 80)
            grade = "B";
        else if (score >= 70)
            grade = "C";
        else
            grade = "F";

        Console.WriteLine($"ពិន្ទុសរុប: {grade}");
    }
}
```

១.៦ ប្រភេទទិន្នន័យ និងអថេរ

(Data Types and Variables)

ក. Value Types និង Reference Types

នៅក្នុង C# មានប្រភេទទិន្នន័យចម្បង 2 គឺ Value Types និង Reference Types។

Value Types:

Value types រក្សាទុកតម្លៃផ្ទាល់នៅក្នុងសតិសារ។ នៅពេលកំណត់ value type មួយទៅ variable ផ្សេង តម្លៃត្រូវបាន copy។

```
int a = 10;
int b = a;   // copy តម្លៃ
b = 20;      // ផ្លាស់ប្តូរតែ b
Console.WriteLine(a);  // Output: 10 (a មិនផ្លាស់ប្តូរ)
Console.WriteLine(b);  // Output: 20
```

Reference Types:

Reference types រក្សាទុកអាសយដ្ឋាន (address) ទៅកាន់វត្ថុនៅក្នុងសតិសារ។ នៅពេលកំណត់ reference type មួយទៅ variable ផ្សេង អាសយដ្ឋានត្រូវបាន copy។

```
int[] arr1 = { 1, 2, 3 };
int[] arr2 = arr1;  // copy reference
arr2[0] = 100;      // ផ្លាស់ប្តូរតាមរយៈ arr2

Console.WriteLine(arr1[0]);  // Output: 100 (arr1 ផ្លាស់ប្តូរដែរ)
```

ខ. Basic Data Types

ប្រភេទទិន្នន័យមូលដ្ឋាននៅក្នុង C#:

ប្រភេទ	ទំហំ	ជួរ	ឧទាហរណ៍
byte	8-bit	0 to 255	byte age = 25;
sbyte	8-bit	-128 to 127	sbyte temp = -10;
short	16-bit	-32,768 to 32,767	short year = 2024;
ushort	16-bit	0 to 65,535	ushort port = 8080;
int	32-bit	-2.1B to 2.1B	int count = 100;
uint	32-bit	0 to 4.2B	uint id = 123456;
long	64-bit	Very large range	long bigNum = 999999;
ulong	64-bit	0 to very large	ulong huge = 999999;
float	32-bit	±1.5e−45 to ±3.4e38	float pi = 3.14f;
double	64-bit	±5.0e−324 to ±1.7e308	double price = 99.99;
decimal	128-bit	±1.0e-28 to ±7.9e28	decimal money = 100.50m;
char	16-bit	Unicode character	char letter = 'A';
bool	1-bit	true or false	bool isValid = true;
string	Variable	Text	string name = "សុខា";

ការជ្រើសរើសប្រភេទទិន្នន័យ:

- int: សម្រាប់លេខគត់ទូទៅ
- double: សម្រាប់លេខទសភាគ (calculations)
- decimal: សម្រាប់លុយកាក់ និងលេខទសភាគត្រឹមត្រូវខ្ពស់
- string: សម្រាប់អត្ថបទ
- bool: សម្រាប់តម្លៃ true/false
- char: សម្រាប់តួអក្សរតែមួយ

គ. Variable Declaration និង Initialization

Variable គឺជាឈ្មោះដែលយើងប្រើដើម្បីរក្សាទុកតម្លៃនៅក្នុងសតិសារ។

```
// Declaration
int age;            // ប្រកាសតែប៉ុណ្ណោះ
age = 25;           // Initialization

// Declaration + Initialization
int score = 100;
string name = "សុខា";
bool isStudent = true;

// Multiple declarations
int x, y, z;
x = 1;
y = 2;
z = 3;

// Multiple declarations with initialization
int a = 1, b = 2, c = 3;

// var keyword (type inference)
var number = 10;        // compiler ដឹងថា int
var text = "Hello";     // compiler ដឹងថា string
var price = 99.99;      // compiler ដឹងថា double
```

Naming Rules និង Conventions:

- ត្រូវចាប់ផ្តើមដោយអក្សរ ឬ underscore (_)

- អាចមានអក្សរ លេខ និង underscores
- មិនអាចប្រើ keywords របស់ C# (int, string, class, etc.)
- Case-sensitive (age និង Age ខុសគ្នា)
- ប្រើ camelCase (myVariable, userName)
- ប្រើឈ្មោះមានន័យ (age ជាជាង x)
- ជៀសវាងឈ្មោះវែងពេក

ឧទាហរណ៍ល្អ និងមិនល្អ:

```
// ល្អ
int studentAge;
string firstName;
double totalPrice;
bool isValid;

// មិនល្អ
int x;              // មិនមានន័យ
string s;           // ខ្លីពេក
int 123abc;         // ចាប់ផ្តើមដោយលេខ (error!)
int my-variable;    // មាន hyphen (error!)
string class;       // keyword (error!)
```

៤. Constants និង Type Casting

Constants:

Constants គឺ variables ដែលមិនអាចផ្លាស់ប្តូរតម្លៃបានបន្ទាប់ពីកំណត់។

```
// ប្រកាស constant
const double PI = 3.14159;
const int MAX_STUDENTS = 30;
const string COMPANY_NAME = "ABC Corp";

// មិនអាចផ្លាស់ប្តូរ
// PI = 3.14;  // Error!

// ការប្រើប្រាស់
double radius = 5;
```

```
double area = PI * radius * radius;
Console.WriteLine($"Area: {area}");
```

Naming Convention សម្រាប់ Constants:

ប្រើអក្សរធំទាំងអស់ជាមួយ underscores (UPPER_CASE)

Type Casting:

Type Casting គឺការបម្លែងតម្លៃពីប្រភេទមួយទៅប្រភេទមួយទៀត។

១. Implicit Casting (ស្វ័យប្រវត្តិ):

```
// ពីតូចទៅធំ (no data loss)
int myInt = 9;
double myDouble = myInt;  // int -> double

Console.WriteLine(myInt);      // 9
Console.WriteLine(myDouble);   // 9.0

// លំដាប់: byte -> short -> int -> long -> float -> double
```

២. Explicit Casting (manual):

```
// ពីធំទៅតូច (possible data loss)
double myDouble = 9.78;
int myInt = (int)myDouble;  // double -> int

Console.WriteLine(myDouble);  // 9.78
Console.WriteLine(myInt);     // 9 (truncated)

// ប្រយ័ត្ន: អាចបាត់បង់ទិន្នន័យ
double big = 1234567890.99;
int small = (int)big;
Console.WriteLine(small);  // overflow!
```

៣. Type Conversion Methods:

```
// Convert class
string numStr = "123";
int num = Convert.ToInt32(numStr);
double dbl = Convert.ToDouble("45.67");
bool flag = Convert.ToBoolean("true");
```

```
// Parse methods
int n1 = int.Parse("100");
double n2 = double.Parse("3.14");
bool n3 = bool.Parse("false");

// TryParse (សុវត្ថិភាពជាង)
string input = "abc";
int result;
if (int.TryParse(input, out result))
{
    Console.WriteLine("Success: " + result);
}
else
{
    Console.WriteLine("មិនអាចបម្លែងបាន!");
}
```

ឧទាហរណ៍ពេញលេញ:

```
using System;

class Program
{
    static void Main()
    {
        // Constants
        const double TAX_RATE = 0.10;

        // Variables
        Console.Write("បញ្ចូលតម្លៃផលិតផល: $");
        string input = Console.ReadLine();

        // Type conversion
        double price;
        if (double.TryParse(input, out price))
        {
            // គណនា
            double tax = price * TAX_RATE;
            double total = price + tax;

            // បង្ហាញលទ្ធផល
            Console.WriteLine($"តម្លៃ: ${price:F2}");
```

```
            Console.WriteLine($"ពន្ធ ({TAX_RATE:P0}): ${tax:F2}");
            Console.WriteLine($"សរុប: ${total:F2}");
        }
        else
        {
            Console.WriteLine("Input មិនត្រឹមត្រូវ!");
        }
    }
}
```

១.៧ ប្រតិបត្តិករ និង Expressions

(Operators and Expressions)

Operators គឺជានិមិត្តសញ្ញាពិសេសដែលប្រាប់ឱ្យកម្មវិធីធ្វើប្រតិបត្តិការជាក់លាក់មួយ។ Expression គឺជាការរួមបញ្ចូលគ្នានៃ values, variables, និង operators ដែលផលិតតម្លៃ។

ក. Arithmetic Operators

Operators សម្រាប់គណិតវិទ្យា:

```
int a = 10;
int b = 3;

// Addition (+)
int sum = a + b;            // 13
Console.WriteLine($"{a} + {b} = {sum}");

// Subtraction (-)
int difference = a - b;     // 7
Console.WriteLine($"{a} - {b} = {difference}");

// Multiplication (*)
int product = a * b;        // 30
Console.WriteLine($"{a} * {b} = {product}");

// Division (/)
int quotient = a / b;       // 3 (integer division)
Console.WriteLine($"{a} / {b} = {quotient}");

double exactDivision = (double)a / b;  // 3.333...
Console.WriteLine($"{a} / {b} = {exactDivision:F2}");
```

```
// Modulus (%) - remainder
int remainder = a % b;      // 1
Console.WriteLine($"{a} % {b} = {remainder}");
```

Increment និង Decrement Operators:

```
int x = 5;

// Increment (++)
x++;        // x = x + 1; (x becomes 6)
++x;        // x = x + 1; (x becomes 7)

// Decrement (--)
x--;        // x = x - 1; (x becomes 6)
--x;        // x = x - 1; (x becomes 5)

// Prefix vs Postfix
int a = 5;
int b = a++;    // b = 5, a = 6 (postfix: use then increment)
int c = ++a;    // c = 7, a = 7 (prefix: increment then use)

Console.WriteLine($"a={a}, b={b}, c={c}");  // a=7, b=5, c=7
```

២. Comparison Operators

Operators សម្រាប់ប្រៀបធៀប (return bool):

```
int a = 10;
int b = 20;

// Equal to (==)
bool isEqual = (a == b);          // false
Console.WriteLine($"{a} == {b}: {isEqual}");

// Not equal to (!=)
bool isNotEqual = (a != b);       // true
Console.WriteLine($"{a} != {b}: {isNotEqual}");

// Greater than (>)
bool isGreater = (a > b);         // false
Console.WriteLine($"{a} > {b}: {isGreater}");

// Less than (<)
bool isLess = (a < b);            // true
Console.WriteLine($"{a} < {b}: {isLess}");

// Greater than or equal (>=)
bool isGreaterEqual = (a >= b);  // false
Console.WriteLine($"{a} >= {b}: {isGreaterEqual}");
```

```
// Less than or equal (<=)
bool isLessEqual = (a <= b);      // true
Console.WriteLine($"{a} <= {b}: {isLessEqual}");
```

គ. Logical Operators

Operators សម្រាប់ logic (ប្រើជាមួយ bool):

```
bool a = true;
bool b = false;

// AND (&&) - ត្រូវតែទាំងពីរ true
bool result1 = a && b;      // false
Console.WriteLine($"{a} && {b} = {result1}");

// OR (||) - យ៉ាងហោចណាស់មួយ true
bool result2 = a || b;      // true
Console.WriteLine($"{a} || {b} = {result2}");

// NOT (!) - ផ្ទុយ
bool result3 = !a;          // false
Console.WriteLine($"!{a} = {result3}");

// ឧទាហរណ៍ប្រើប្រាស់
int age = 25;
bool hasLicense = true;
bool canDrive = (age >= 18) && hasLicense;  // true
Console.WriteLine($"អាចបើកបរ: {canDrive}");
```

Truth Tables:

ឈ្មោះនឹង AND (&&):

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

ឈ្នាប់ឬ OR (||):

A	B	A || B
true	true	true
true	false	true
false	true	true
false	false	false

៤. Assignment Operators

Operators សម្រាប់កំណត់តម្លៃ:

```
int x = 10;

// Simple assignment
x = 5;      // x = 5

// Compound assignment
x += 3;     // x = x + 3;   (8)
x -= 2;     // x = x - 2;   (6)
x *= 4;     // x = x * 4;   (24)
x /= 3;     // x = x / 3;   (8)
x %= 5;     // x = x % 5;   (3)

// ឧទាហរណ៍ពេញលេញ
int score = 0;
score += 10;    // រក score 10
score += 15;    // រក score 15 ទៀត
score -= 5;     // ដក 5
score *= 2;     // គុណនឹង 2
Console.WriteLine($"Final score: {score}");  // 40
```

Operator Precedence:

លំដាប់អាទិភាពនៃការប្រតិបត្តិ operators (ពីខ្ពស់ទៅទាប):

1. ( ), [] - Parentheses និង array access
2. !, ++, -- - Unary operators
3. *, /, % - Multiplication, division, modulus
4. +, - - Addition និង subtraction

5. <, >, <=, >= - Comparison
6. ==, != - Equality
7. && - Logical AND
8. || - Logical OR
9. = - Assignment

```
// ឧទាហរណ៍
int result = 10 + 5 * 2;     // 20 (not 30) - * មុន +
int result2 = (10 + 5) * 2;  // 30 - () មុនគេ

// ប្រើ () ដើម្បីឱ្យច្បាស់
int a = 5;
int b = 3;
int c = 2;
int x = a + b * c;       // 11
int y = (a + b) * c;     // 16

Console.WriteLine($"x = {x}, y = {y}");
```

ឧទាហរណ៍ពេញលេញ - Calculator Program:

```
using System;

class Calculator
{
    static void Main()
    {
        Console.WriteLine("=== កម្មវិធីគណនា ===\n");

        // Input
        Console.Write("បញ្ចូលលេខទី១: ");
        double num1 = double.Parse(Console.ReadLine());

        Console.Write("បញ្ចូលលេខទី២: ");
        double num2 = double.Parse(Console.ReadLine());

        Console.Write("ជ្រើសរើសប្រតិបត្តិការ (+, -, *, /, %): ");
```

```
            char operation = Console.ReadKey().KeyChar;
            Console.WriteLine("\n");

            // គណនា
            double result = 0;
            bool validOperation = true;

            switch (operation)
            {
                case '+':
                    result = num1 + num2;
                    break;
                case '-':
                    result = num1 - num2;
                    break;
                case '*':
                    result = num1 * num2;
                    break;
                case '/':
                    if (num2 != 0)
                        result = num1 / num2;
                    else
                    {
                        Console.WriteLine("Error: មិនអាចចែកនឹង 0!");
                        validOperation = false;
                    }
                    break;
                case '%':
                    result = num1 % num2;
                    break;
                default:
                    Console.WriteLine("ប្រតិបត្តិការមិនត្រឹមត្រូវ!");
                    validOperation = false;
                    break;
            }

            // Output
            if (validOperation)
            {
                Console.WriteLine($"លទ្ធផល: {num1} {operation} {num2} =
{result:F2}");
            }

            Console.ReadLine();
        }
    }
```

សង្ខេប (Summary)

នៅក្នុងមេរៀនទី១នេះ យើងបានរៀន៖

- គោលការណ៍មូលដ្ឋាននៃការសរសេរកម្មវិធី និងប្រភេទភាសាកម្មវិធី
- ប្រវត្តិ និងការវិវឌ្ឍន៍នៃភាសា C#
- ការដំឡើង និងការប្រើប្រាស់ Visual Studio
- រចនាសម្ព័ន្ធកម្មវិធី C# មូលដ្ឋាន
- Namespaces, using directives, និង Main method
- Console Input/Output operations
- ប្រភេទទិន្នន័យ Value types និង Reference types
- Variable declaration, initialization, និង naming conventions
- Constants និង Type casting
- Arithmetic, Comparison, Logical និង Assignment operators
- Operator precedence និង expressions

លំហាត់អនុវត្ត (Exercises)

ផ្នែកទី១៖ សំណួរពហុជ្រើសរើស

១. C# ត្រូវបានបង្កើតឡើងដោយក្រុមហ៊ុនអ្វី?

A) Google

B) Microsoft

C) Apple

D) Oracle

២. Main method គឺជាអ្វី?

A) ចំណុចចាប់ផ្តើមនៃកម្មវិធី

B) ចុងក្រោយនៃកម្មវិធី

C) មិនសំខាន់

D) optional

៣. Console.ReadLine( ) return ប្រភេទអ្វី?

A) int

B) double

C) string

D) bool

៤. តើ operator មួយណាមានអាទិភាពខ្ពស់បំផុត?

A) +

B) *

C) ( )

D) ==

៥. តើ value type មួយណាខាងក្រោម?

A) string

B) array

C) int

D) class

ផ្នែកទី២៖ លំហាត់សរសេរកូដ

១. សរសេរកម្មវិធីដែលស្នើឱ្យអ្នកប្រើបញ្ចូលឈ្មោះ និងបង្ហាញសារស្វាគមន៍។

២. សរសេរកម្មវិធីគណនាផ្ទៃរង្វង់ (Area = π × r²)។

៣. សរសេរកម្មវិធីបម្លែងសីតុណ្ហភាពពី Celsius ទៅ Fahrenheit (F = C × 9/5 + 32)។

៤. សរសេរកម្មវិធីពិនិត្យថាលេខមួយគឺ even ឬ odd។

៥. សរសេរកម្មវិធី calculator សាមញ្ញដែលអាចធ្វើប្រតិបត្តិការ +, -, *, /។

ផ្នែកទី៣៖ លំហាត់ប្រឈម (Challenge)

១. បង្កើតកម្មវិធីគណនាពន្ធលើតម្លៃផលិតផល (Price + Tax = Total)។

២. បង្កើតកម្មវិធីគណនា BMI (Body Mass Index) = weight(kg) / height(m)²។

៣. បង្កើតកម្មវិធីគណនា compound interest: A = P(1 + r/n)^(nt)

ឯកសារយោង (References)

- Microsoft C# Documentation: https://docs.microsoft.com/en-us/dotnet/csharp/
- C# Programming Guide: https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/
- Visual Studio Documentation: https://docs.microsoft.com/en-us/visualstudio/
- .NET Documentation: https://docs.microsoft.com/en-us/dotnet/
- C# Language Specification: https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/